

MemoryCare AI Development Guide

Current architecture, safe preparation boundaries, and developer handoff

IMPLEMENTED

Purpose. This guide describes the existing MemoryCare application and the safe preparation work needed before a friend connects two separate AI systems: a video-analysis API and a local analytics service. The application is preserved as an offline-first dementia-care demonstration. Neither AI system is implemented by this guide.

Reading the labels. **IMPLEMENTED** means the repository or deployed application already provides the behavior.

PLACEHOLDER means a documented contract or adapter boundary is intended but does not contact a real AI system. **FUTURE IMPLEMENTATION** means a later developer must make a product or security decision before coding it.

1. Product and current system

MemoryCare is an elderly-friendly, offline-first progressive web application for gentle cognitive games, daily reminders, familiar people, emergency contacts, progress trends, and caregiver views. Its language and interaction model are designed for large touch targets, voice guidance, reduced motion, high contrast, and patient-friendly wording. It explicitly avoids medical diagnosis claims.

The current application is a React 18 and TypeScript single-page application built with Vite. The browser can run in Guest Mode without Supabase. When configured, Supabase provides authentication, PostgreSQL data, Storage, Row Level Security, an OTP Edge Function, and central diagnostic events.

2. Current implementation inventory

Area	Status	What the code currently does
React and routing	IMPLEMENTED	React 18, TypeScript, Vite, react-router-dom, route guards, separate patient and caregiver layouts.
Patient experience	IMPLEMENTED	Home, games, game play, reminders, progress, profiles, people, emergency, settings, language selection.
Caregiver experience	IMPLEMENTED	Overview, progress, alerts, patient profile, reminders, emergency, people, settings.
Authentication	IMPLEMENTED	Supabase browser sessions, username-to-internal-email mapping, password flow, Google OAuth hook, recovery route, role selection.
Guest Mode	IMPLEMENTED	Stable local patient namespace, seeded demo content, IndexedDB/localStorage, no Supabase session.
Supabase data	IMPLEMENTED	Patients, caregivers, links, people, games, sessions, metrics, difficulty history, reminders, emergency data, RLS.
Storage	IMPLEMENTED	Private patient-media bucket and patient-scoped Storage policies for existing photos/voice media.
Diagnostics	IMPLEMENTED	Sanitized client event queue, system_events table, Edge logging helper, admin error dashboard.
Video AI	PLACEHOLDER	No external video analysis is performed. A future provider boundary will be added without exposing secrets.
Local AI	PLACEHOLDER	No local model or endpoint is contacted. A future configurable adapter will accept aggregate application data.

3. Repository map

src/	
App.tsx	Route tree and access guards
components/	Shared UI, layouts, state and navigation
data/	Game definitions and local seed data
games/	Individual game implementations
hooks/	Settings, progress, reminders, voice, online state
i18n/	Language provider and locale dictionaries
lib/	Supabase client and diagnostics
pages/	Patient, caregiver, auth, admin and onboarding routes
services/	Auth, data, storage, sync, reporting and diagnostics
styles/	Design tokens, base, component, game and mobile CSS
types/	Shared TypeScript domain types
utils/	Date, profile and general helpers
supabase/	
schema.sql	Foundation schema and RLS policies
migrations/	Incremental repairs and feature migrations
functions/	auth-otp Edge Function and shared logging
docs/	Existing authentication notes plus future handoff docs
public/	PWA manifest, icons, service worker and image assets

4. Routes and access model

The route tree is intentionally explicit. Public routes include the landing page, authentication, password recovery, language selection, choose-role, and the admin diagnostics path. Patient routes sit behind `RequireOnboarded`, which requires an authenticated or Guest Mode session. Caregiver routes sit behind `RequireCaregiver`, which additionally requires the caregiver role. The admin page performs a separate server-authorized check through `is_my_system_admin` and RLS.

Patient routes: `/home`, `/games`, `/games/:gameId`, `/reminders`, `/today`, `/emergency`, `/progress`, `/settings`, `/profiles`, `/people`.

Caregiver routes: `/caregiver`, `/caregiver/progress`, `/caregiver/alerts`, `/caregiver/patient`, `/caregiver/settings`, `/caregiver/reminders`, `/caregiver/emergency`, `/caregiver/people`.

5. Supabase architecture

Supabase is optional at runtime. The browser client is created only when the public URL and anon/publishable key are present. The client enables session persistence, automatic refresh, and URL detection. The frontend accepts only public configuration; service-role credentials are not accepted through Vite variables.

The database separates identity from application data. `auth.users` is managed by Supabase Auth. `public.profiles` stores role, display name, language, and onboarding state. `patients` represent the patient owner context. `caregivers` and `caregiver_patient` express authorized caregiver access. `person_memories`, `reminders`, `game_sessions`, `game_metrics`, `adaptive_difficulty_history`, `emergency_contacts`, and `patient_emergency_info` are patient-scoped feature tables. The private schema contains authorization helpers, rate-limit state, OTP challenge state, and explicitly provisioned admin users.

RLS is the primary authorization boundary. Patient access is based on `auth.uid` ownership; caregiver access requires an active `caregiver_patient` link. System events are insertable by normal clients but readable and resolvable only by explicitly provisioned system administrators.

6. Authentication, storage and privacy

The username/password UI deterministically maps a valid username to an internal email-like identity under `users.memorycare.app`. Google sign-in remains a Supabase provider configuration. First sign-in can require role selection. Guest Mode never establishes a Supabase session. Passwords, access tokens, refresh tokens, OTPs and API keys must never be logged or placed in frontend source.

Existing patient media uses the private patient-media Storage bucket and a patient-id prefix. Existing People functionality can reference photo and voice recording paths. Signed URLs, upload progress, cancellation, large-video handling, and video-specific retention rules are not currently implemented.

7. Existing admin and error monitoring

AdminErrors is an existing diagnostics console, not an AI operations console. It checks explicit admin provisioning, filters and groups system_events, shows severity and request context, and allows an administrator to mark events resolved. The UI currently displays service labels such as Frontend, Auth, Database, Storage and Edge Functions; only the first three are currently represented as operational without a real health check. Future AI status must use Unknown until a live health check confirms a state.

The central client errorLogger redacts JWT-like tokens, emails, secret-shaped fields and oversized values; deduplicates events; rate-limits local capture; stores a bounded local queue; and flushes to Supabase when available. Global error and unhandled rejection listeners are installed at startup. The Edge Function has a shared logger for server-side events. The same system should be reused by AI adapters.

8. Planned AI architecture

The two AI systems must remain separate. Video AI receives an explicitly submitted video reference and returns a structured analysis result. Local AI receives a deliberately minimized aggregate data snapshot and returns trends or recommendations. Neither system should be called implicitly from a page render, and neither placeholder should transmit data when its endpoint is absent.

PLACEHOLDER

Video AI boundary. The future provider should expose submit, get status, and optionally cancel/retry operations. The frontend should store only a safe job reference and status. A Supabase Edge Function or other server-side boundary should hold any private provider key and create signed Storage URLs only when authorized.

PLACEHOLDER

Local AI boundary. The future provider should accept a configurable endpoint, a correlation ID, an authorized patient scope, and aggregate metrics. It must not assume localhost or a fixed IP in production. The friend later supplies the endpoint and authentication method through server-side configuration or a secure proxy.

9. AI contracts

These contracts are intentionally provider-neutral. They describe the shape the future adapter should validate. They are not claims that a provider exists today.

Video request and response

```
VideoAnalysisRequest
{
  "jobId": "uuid",
  "videoId": "uuid",
  "storagePath": "patient-id/videos/example.mp4",
  "patientId": "uuid",
  "analysisType": "engagement | routine | custom",
  "metadata": { "durationSeconds": 120, "contentType": "video/mp4" },
  "correlationId": "uuid"
}

VideoAnalysisResult
{
  "analysisId": "uuid",
  "jobId": "uuid",
  "status": "queued | processing | completed | failed",
  "summary": "provider-generated summary",
  "metrics": {},
  "confidence": 0.0,
  "timestamps": [],
  "provider": "friend-video-service",
  "model": "model-name",
  "createdAt": "2026-01-01T00:00:00Z",
  "completedAt": null,
  "error": null
}
```

Local request and response

```
LocalAnalysisRequest
{
  "jobId": "uuid",
  "patientId": "uuid",
  "period": { "from": "2026-01-01", "to": "2026-01-31" },
  "metrics": { "sessions": [], "reminders": [], "trend": {} },
  "correlationId": "uuid"
}

LocalAnalysisResult
{
  "analysisId": "uuid",
  "status": "queued | processing | completed | failed",
  "summary": "structured trend summary",
  "signals": [],
  "recommendations": [],
  "provider": "friend-local-ai",
  "model": "model-name",
  "createdAt": "2026-01-01T00:00:00Z",
  "error": null
}
```

10. Job lifecycle and data structure

AI work must be asynchronous. The proposed lifecycle is queued -> processing -> completed or failed. A retry must be explicit and bounded. Each job should retain a provider, source type, status, timestamps, retry count, correlation ID, and safe error information. JSONB is appropriate for model-specific output, but raw private media or unnecessary personal information should not be duplicated in results.

FUTURE IMPLEMENTATION

A minimal future schema may include `ai_analysis_jobs` and `ai_analysis_results`, with a separate video reference table only if the existing media model cannot safely represent it. Before adding tables, inspect deployed schema and migrations to avoid duplicate concepts. RLS must restrict rows by patient authorization, and admin reads must use explicit admin authorization.

11. Admin monitoring extension

Extend `AdminErrors` rather than replacing it. Add an AI operations section only after the job tables and server health checks exist. Video AI and Local AI should show Operational, Degraded, Failing, or Unknown. Unknown is the correct initial state when no health check has run. The section can later show recent jobs, pending jobs, failures, provider/model, timestamps, retry counts and correlation IDs.

12. Configuration and secrets

Public browser configuration belongs in `VITE_*` variables and may include the Supabase URL and public anon/publishable key. Private provider keys must not use `VITE_*`. They belong in Supabase Edge Function secrets, Vercel server-side environment variables, or another controlled backend. A browser-visible endpoint may be public only if it authenticates requests safely and does not reveal a secret.

The future `.env.example` should contain placeholders only:

```
# Public browser configuration
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=your-anon-or-publishable-key

# Future server-side configuration - do not prefix with VITE_
VIDEO_AI_API_URL=
VIDEO_AI_API_KEY=server-side-only
LOCAL_AI_API_URL=
LOCAL_AI_API_KEY=server-side-only
```

13. Mobile and performance plan

The current design already has mobile CSS, responsive breakpoints, accessibility toggles, a service worker, lazy PDF import, and patient-friendly touch controls. A safe optimization pass should measure rather than redesign. Test 320, 360, 390, 414 and desktop widths for every major route. Check horizontal overflow, fixed navigation overlap, sheets and keyboards, focus order, large text, jumbo buttons, high contrast and reduced motion.

Potential safe improvements include route-level lazy imports, stable memoization only where profiling justifies it, request deduplication, abortable long requests, media size/type validation, and explicit loading states. Do not hide overflow to mask a layout defect, and do not add subscriptions without cleanup. Changes should be verified against Guest Mode, authenticated mode and caregiver mode.

14. Media and video preparation

Existing media is limited to patient-scoped photos and voice references. Video upload and AI submission should be explicit user actions. A later implementation should validate file type and size before upload, show progress, support cancellation and retry, use private Storage paths, generate short-lived signed URLs, and record a job only after the upload is complete. It must never automatically send private video to an external service.

15. Developer handoff on another computer

Use the repository as the source of truth. Do not copy production secrets into the repository. The friend should clone the repository, install dependencies with the lockfile, create `.env.local` from `.env.example`, configure the public Supabase values, apply migrations, and run the development server. For Edge Functions, configure Supabase CLI secrets separately. For AI, implement the documented provider behind the adapter and expose only a safe server-side route to the frontend.

Suggested sequence

- Clone the repository and run `npm install`.
- Copy `.env.example` to `.env.local` and add public Supabase values.
- Run `npm run build` before changing the project.
- Apply schema and migrations in Supabase; verify RLS and Storage policies.
- Run `npm run dev` and test Guest Mode first.
- Add the local AI endpoint only through the documented server-side boundary.
- Deploy the frontend to Vercel with public variables only.
- Deploy and configure Edge Functions separately; never commit secrets.
- Test admin provisioning and system event visibility with a disposable account.

16. Testing and release gates

The existing build command performs TypeScript checking and a Vite production build. A future test script should add unit tests for adapters, schema validators, sanitization, job transitions and route guards. Live verification should cover every route family, Guest Mode, authentication, Supabase access, admin denial, admin success, language selection, accessibility toggles, mobile widths and placeholder AI behavior with no endpoint configured.

A placeholder provider should return a typed configuration error such as `AI_PROVIDER_NOT_CONFIGURED`, never silently claim success, never crash a page, and never transmit a request when no endpoint is configured. Error events must include a safe operation and correlation ID but never include passwords, tokens, keys, private media or unnecessary patient details.

17. Known limitations and explicit future work

Area	Status	What the code currently does
Actual AI models	FUTURE IMPLEMENTATION	No video or local model exists in this repository. A friend must supply and operate the model service.
AI provider adapters	PLACEHOLDER	Contracts and boundaries are planned; implementation must validate request/response data and use safe server-side secrets.
AI database tables	FUTURE IMPLEMENTATION	Add only after confirming the deployed schema and deciding whether existing tables can represent jobs safely.
AI admin monitoring	FUTURE IMPLEMENTATION	Extend the existing diagnostics page after health checks and job tables exist.
Video upload pipeline	FUTURE IMPLEMENTATION	Existing patient media references do not yet provide large-video validation, cancellation or signed-URL workflow.
Cross-device sync	FUTURE IMPLEMENTATION	The current product is a local-device demo even when Supabase features are enabled.
Background alarms	FUTURE IMPLEMENTATION	Browser notifications are best-effort and not guaranteed when the app is closed.
Medical interpretation	NOT A FEATURE	Progress is an activity trend and must not be presented as diagnosis or clinical scoring.

18. Baseline conclusion

The safe path is additive: preserve the current route guards, services, Supabase policies, Guest Mode and accessibility system; add typed AI contracts and adapters behind explicit calls; reuse the existing redacted diagnostics system; and keep all provider credentials server-side. The friend receiving this project should implement only the provider-specific work and the operational controls described above, rather than reverse-engineering the application.